RECUEIL DE REVISION - JAVA / JDBC / JEE

Objectif du document

- Réviser rapidement les notions les plus fréquentes dans un examen Java/JDBC/JEE.
- Retenir la syntaxe minimale, les pièges classiques et les modèles de code à adapter.
- Préparer les réponses écrites : définitions courtes, différences, étapes et exemples.

Méthode de révision conseillée

- Lire chaque fiche puis recopier les modèles de code à la main.
- Comprendre les signatures : paramètres, type de retour, exceptions.
- Avant l'examen : mémoriser les blocs essentiels JDBC, Socket, Thread, Servlet/JSP/Filter.

1. Redéfinition: equals, compareTo, clone

1.1 equals()

Idée principale

- equals() sert à comparer le contenu logique de deux objets, pas leurs adresses mémoire.

```
class Produit {
    private int id;
    private String nom;

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null) return false;
        if (getClass() != obj.getClass()) return false;
        Produit p = (Produit) obj;
        return this.id == p.id;
    }
}
```

Point	Explication
this == obj	Même référence mémoire : forcément égal.
obj == null	Un objet réel n'est jamais égal à null.
getClass()	On vérifie que les deux objets sont de la même classe.
cast	Après vérification, on convertit Object vers la classe réelle.

1.2 compareTo()

Idée principale

- compareTo() sert au tri naturel. Il retourne un entier négatif, zéro ou positif.
- La classe doit implémenter Comparable<T>.

```
class Produit implements Comparable<Produit> {
    private String nom;
    private double prix;

    public int compareTo(Produit p) {
        return Double.compare(this.prix, p.prix); // tri croissant par prix }}
```

Cas particulier : compareTo(Object)

```
public int compareTo(Object o) {

    Personne p = (Personne) o;
    return this.nom.compareTo(p.nom);
}
```

Piège fréquent

- compareTo() ne sert pas à tester seulement l'égalité. Il sert à ordonner.
- Pour les objets, on préfère Comparable<Produit> plutôt que Comparable sans type générique.

1.3 clone()

Idée principale

- clone() crée une copie de l'objet.
- La classe doit implémenter Cloneable.
- La copie peut être superficielle ou profonde selon les attributs.

```
class Employe implements Cloneable {
    private int id;
    private String nom;

    public Employe clone() {
        try {
            return (Employe) super.clone();
        } catch (CloneNotSupportedException e) {
            return null;
        }
    }
}
```

2. ArrayList<E>:

- add(E e) : ajouter un élément
- remove(int index) / remove(Object o) : supprimer
- set(int index, E e) : modifier un élément
- sort(Comparator c) : trier la liste

Exemple :

```
ArrayList<String> list = new ArrayList<>();
list.add("B");
list.add("A");
list.set(0, "C");
list.remove("A");
Collections.sort(list);
```

3. Exceptions

Définition

- Une exception est un événement qui interrompt le déroulement normal du programme.
- Elle se traite avec try/catch/finally ou se propage avec throws.

Type	Exemples	Remarque
Checked exception	IOException, SQLException	Obligatoire : traiter ou déclarer throws.
Unchecked exception	NullPointerException, ArithmeticException	Pas obligatoire, mais il faut éviter les causes.
Error	OutOfMemoryError	Problème grave, généralement non traité.

```
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Division impossible");
} finally {
    System.out.println("Bloc toujours exécuté");
}
```

3.1 try-with-resources

```
try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    String ligne;
    while ((ligne = br.readLine()) != null) {
        System.out.println(ligne);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

À retenir

- try-with-resources ferme automatiquement les ressources.
- Très recommandé pour fichiers, connexions, statements, ResultSet.

4. Threads : Thread et Runnable

Définition

- Un thread est un fil d'exécution indépendant.
- Il permet d'exécuter plusieurs tâches presque en parallèle.

4.1 Hériter de Thread

```
class MonThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread : " + i);
        }
    }
}

public class Test {
    public static void main(String[] args) {
        MonThread t = new MonThread();
        t.start(); // lance un nouveau thread
    }
}
```

4.2 Implémenter Runnable

```
class MaTache implements Runnable {
    public void run() {
        System.out.println("Tâche exécutée");
    }
}

Thread t = new Thread(new MaTache());
t.start();
```

Méthode	Rôle
start()	Crée un nouveau thread puis appelle run().
run()	Contient le code exécuté par le thread. Si on l'appelle directement, pas de nouveau thread.
sleep(ms)	Met en pause le thread courant.
join()	Attend la fin d'un autre thread.
synchronized	Protège une ressource partagée.

```
class Compteur {
    private int n = 0;
    public synchronized void incrementer() {
        n++;
    }
}
```

5. Fichiers : sérialisation, PrintWriter, BufferedReader

5.1 PrintWriter : écrire dans un fichier texte

```
try (PrintWriter pw = new PrintWriter(new FileWriter("notes.txt", true))) {
    pw.println("Bonjour");
    pw.println(2026);
} catch (IOException e) {
    e.printStackTrace();
}
```

5.2 BufferedReader : lire un fichier texte

```
try (BufferedReader br = new BufferedReader(new FileReader("notes.txt"))) {
    String ligne;
    while ((ligne = br.readLine()) != null) {
        System.out.println(ligne);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

5.3 Sérialisation

Définition

- La sérialisation transforme un objet en flux pour l'enregistrer dans un fichier.
- La classe doit implémenter Serializable.

```
class Etudiant implements Serializable {
    private int id;
    private String nom;
}
try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("etudiant.dat"))) {
    out.writeObject(new Etudiant());
}
try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("etudiant.dat"))) {
    Etudiant e = (Etudiant) in.readObject();
} catch (IOException | ClassNotFoundException ex) {  ex.printStackTrace();}
```

6. Réseau : ServerSocket et Socket

Idée principale

- ServerSocket attend les clients côté serveur.
- Socket représente la connexion entre un client et un serveur.
- Les flux d'entrée/sortie permettent d'échanger des messages.

6.1 Serveur TCP simple

```
ServerSocket server = new ServerSocket(5000);
System.out.println("Serveur en attente...");
Socket client = server.accept();

BufferedReader in = new BufferedReader(new InputStreamReader(client.getInputStream()));
```

```
PrintWriter out = new PrintWriter(client.getOutputStream(), true);

String msg = in.readLine();
System.out.println("Client : " + msg);
out.println("Message reçu");

client.close();
server.close();
```

6.2 Client TCP simple

```
Socket socket = new Socket("localhost", 5000);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

out.println("Bonjour serveur");
System.out.println(in.readLine());

socket.close();
```

7. JDBC : connexion, lecture, insertion, modification

Étapes JDBC

- Charger le driver si nécessaire.
- Créer une connexion avec DriverManager.
- Préparer une requête avec PreparedStatement.
- Exécuter la requête.
- Parcourir ResultSet pour SELECT.
- Fermer les ressources.

7.1 Connexion

```
String url = "jdbc:mysql://localhost:3306/gestion";
String user = "root";
String pass = "";

Connection cn = DriverManager.getConnection(url, user, pass);
```

7.2 Lecture : SELECT

```
String sql = "SELECT id, nom, prix FROM produit";
try (Connection cn = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = cn.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        int id = rs.getInt("id");
        String nom = rs.getString("nom");
        double prix = rs.getDouble("prix");
        System.out.println(id + " " + nom + " " + prix);
    }
} catch (SQLException e) { e.printStackTrace();}
```

7.3 Insertion : INSERT

```
String sql = "INSERT INTO produit(nom, prix) VALUES(?, ?)";
try (Connection cn = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = cn.prepareStatement(sql)) {

    ps.setString(1, "Clavier");
    ps.setDouble(2, 120.50);
    int n = ps.executeUpdate();
    System.out.println(n + " ligne insérée");
}
```

7.4 Modification : UPDATE

```
String sql = "UPDATE produit SET prix=? WHERE id=?";
try (Connection cn = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = cn.prepareStatement(sql)) {

    ps.setDouble(1, 150.00);
    ps.setInt(2, 1);
    int n = ps.executeUpdate();
    System.out.println(n + " ligne modifiée");
}
```

Méthode	Utilisation
executeQuery()	Pour SELECT : retourne ResultSet.
executeUpdate()	Pour INSERT, UPDATE, DELETE : retourne le nombre de lignes touchées.
setString, setInt, setDouble	Remplacent les ? de PreparedStatement.
commit / rollback	Gestion des transactions si autoCommit=false.

8. Swing : interface graphique

Composants fréquents

- JFrame : fenêtre principale.
- JLabel : texte.
- JTextField : zone de saisie.
- JButton : bouton.
- JTable : tableau.
- JPanel : conteneur.

```
import javax.swing.*;
import java.awt.*;

public class Fenetre extends JFrame {
    private JTextField txtNom = new JTextField(15);
    private JButton btn = new JButton("Valider");
```

```
    public Fenetre() {
        setTitle("Application Swing");
        setSize(350, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());

        add(new JLabel("Nom :"));
        add(txtNom);
        add(btn);

        btn.addActionListener(e -> {
            JOptionPane.showMessageDialog(this, "Bonjour " + txtNom.getText());
        });
    }

    public static void main(String[] args) {
        new Fenetre().setVisible(true);
    }
}
```

À retenir

- Les événements sont traités avec ActionListener.
- JOptionPane permet d'afficher des messages.
- setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE) ferme l'application.

9. Date, StringBuilder, StringTokenizer

9.1 Date

Classe	Utilisation
java.util.Date	Ancienne classe pour représenter une date/heure.
java.sql.Date	Date compatible avec SQL DATE.

```
// Date actuelle pour SQL
java.sql.Date d = new java.sql.Date(System.currentTimeMillis());
ps.setDate(1, d);

// À partir d'une chaîne
java.sql.Date d2 = java.sql.Date.valueOf("2026-04-27");
```

9.2 StringBuilder

Pourquoi StringBuilder ?

- Il permet de construire une chaîne modifiable.
- Plus efficace que concaténer beaucoup de String avec + dans une boucle.

```
StringBuilder sb = new StringBuilder();
sb.append("Bonjour");
sb.append(" ");
sb.append("Java");
System.out.println(sb.toString());
```

9.3 StringTokenizer

```
String ligne = "Ali;20;DAI";
StringTokenizer st = new StringTokenizer(ligne, ";");
while (st.hasMoreTokens()) {
    System.out.println(st.nextToken());
}
```

Remarque

- StringTokenizer accepte plusieurs séparateurs, par exemple ";,:".
- Aujourd'hui, split() est souvent plus utilisé, mais StringTokenizer reste demandé dans certains examens.

10. JEE : doGet, doPost, JSP, JSTL, doFilter

10.1 Servlet : doGet et doPost

Méthode	Quand l'utiliser ?
doGet()	Afficher une page, faire une recherche, envoyer des paramètres visibles dans URL.
doPost()	Envoyer un formulaire, ajouter/modifier des données, paramètres non visibles dans URL.

```
@WebServlet("/produits")
public class ProduitServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setAttribute("message", "Liste des produits");
        request.getRequestDispatcher("produits.jsp").forward(request, response);
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String nom = request.getParameter("nom");
        // insertion en base via DAO
        response.sendRedirect(request.getContextPath() + "/produits");
    }
}
```

Forward ou Redirect ?

- forward : transfert interne vers JSP, garde les attributs request.
- redirect : nouvelle requête du navigateur, utile après insertion/modification avec PRG.

10.2 JSP + EL

```
<!-- Afficher un attribut envoyé par la servlet -->
<p>${message}</p>
<p>Utilisateur : ${sessionScope.user}</p>
```

10.3 JSTL

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<c:forEach var="p" items="${produits}">
    <tr>
        <td>${p.id}</td>
        <td>${p.nom}</td>
        <td>${p.prix}</td>
    </tr>
</c:forEach>

<c:choose>
    <c:when test="${empty produits}">Aucun produit</c:when>
    <c:otherwise>Produits disponibles</c:otherwise>
</c:choose>
```

10.4 Filter : doFilter

Rôle du Filter

- Intercepter les requêtes avant la servlet/JSP.
- Vérifier l'authentification.
- Appliquer un encodage UTF-8.
- Bloquer ou laisser passer avec chain.doFilter().

```
@WebFilter("/*")
public class AuthFilter implements Filter {
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) res;
        HttpSession session = request.getSession(false);

        String uri = request.getRequestURI();
        boolean pageLogin = uri.endsWith("login.jsp") || uri.endsWith("/login");
        boolean connecte = session != null && session.getAttribute("user") != null;
```

```
        if (connecte || pageLogin) {
            chain.doFilter(req, res);
        } else {
            response.sendRedirect(request.getContextPath() + "/login.jsp");
        }
    }
}
```

Pièges JEE

- Avec Tomcat 10/11, utiliser jakarta.servlet.* et non javax.servlet.*.
- La JSTL moderne utilise souvent uri="jakarta.tags.core".
- request.getContextPath() évite les erreurs de chemin.
- request.getSession(false) évite de créer une session inutile.

11. Fiche synthèse finale

Thème	À mémoriser absolument
equals	Comparer le contenu logique : this==obj, null, getClass, cast, comparaison.
compareTo	Retourne négatif, zéro, positif pour le tri.
clone	Cloneable + super.clone().
Exceptions	try/catch/finally, throws, try-with-resources.
Thread	start() lance un thread ; run() contient le code.
Fichier	BufferedReader pour lire, PrintWriter pour écrire, Serializable pour objet.
Socket	ServerSocket côté serveur, Socket côté client.
JDBC	Connection, PreparedStatement, ResultSet, executeQuery, executeUpdate.
Swing	JFrame, JButton, JTextField, ActionListener.
JEE	Servlet doGet/doPost, JSP/EL/JSTL, Filter doFilter.

Derniers conseils pour l'examen

- Toujours préciser le rôle de chaque classe ou méthode avant le code.
- Utiliser PreparedStatement au lieu de concaténer les paramètres SQL.
- Pour JEE, séparer : Servlet = contrôleur, JSP = affichage, DAO = accès aux données.
- Écrire un code simple, lisible et cohérent avec les signatures demandées.